

ITC Assignment 1 - Report

1. Objective

To implement D-ary Huffman, Shannon-Fano, Shannon, and Shannon-type encoding for a given set of source probabilities. The program computes entropy, expected code length, efficiency, verifies entropy bounds and Kraft inequality, and performs decoding using Huffman encoding.

2. Assumptions

- Input source probabilities sum to 1.
- D does not exceed 9, as the program uses only the digits 0,...,D-1 as D-ary alphabet.
- Huffman decoding is performed only using the Huffman code.

3. Test Case and Output

D = 3

Number of symbols = 15

Probabilities:

S0 = 0.18, S1 = 0.14, S2 = 0.12, S3 = 0.10, S4 = 0.08, S5 = 0.07, S6 = 0.06, S7 = 0.05, S8 = 0.05
S9 = 0.04, S10 = 0.03, S11 = 0.03, S12 = 0.02, S13 = 0.02, S14 = 0.01

--- Performance ---

Entropy (base 3) = 2.2454

Expected Lengths:

Huffman = 2.3000

Shannon-Fano = 2.5100

Shannon = 2.6800

Shannon-type = 2.6800

Efficiency:

Huffman = 0.9763

Shannon-Fano = 0.8946

Shannon = 0.8379

Shannon-type = 0.8379

Entropy Bound Check : $H \leq L < H+1$ satisfied for all methods.

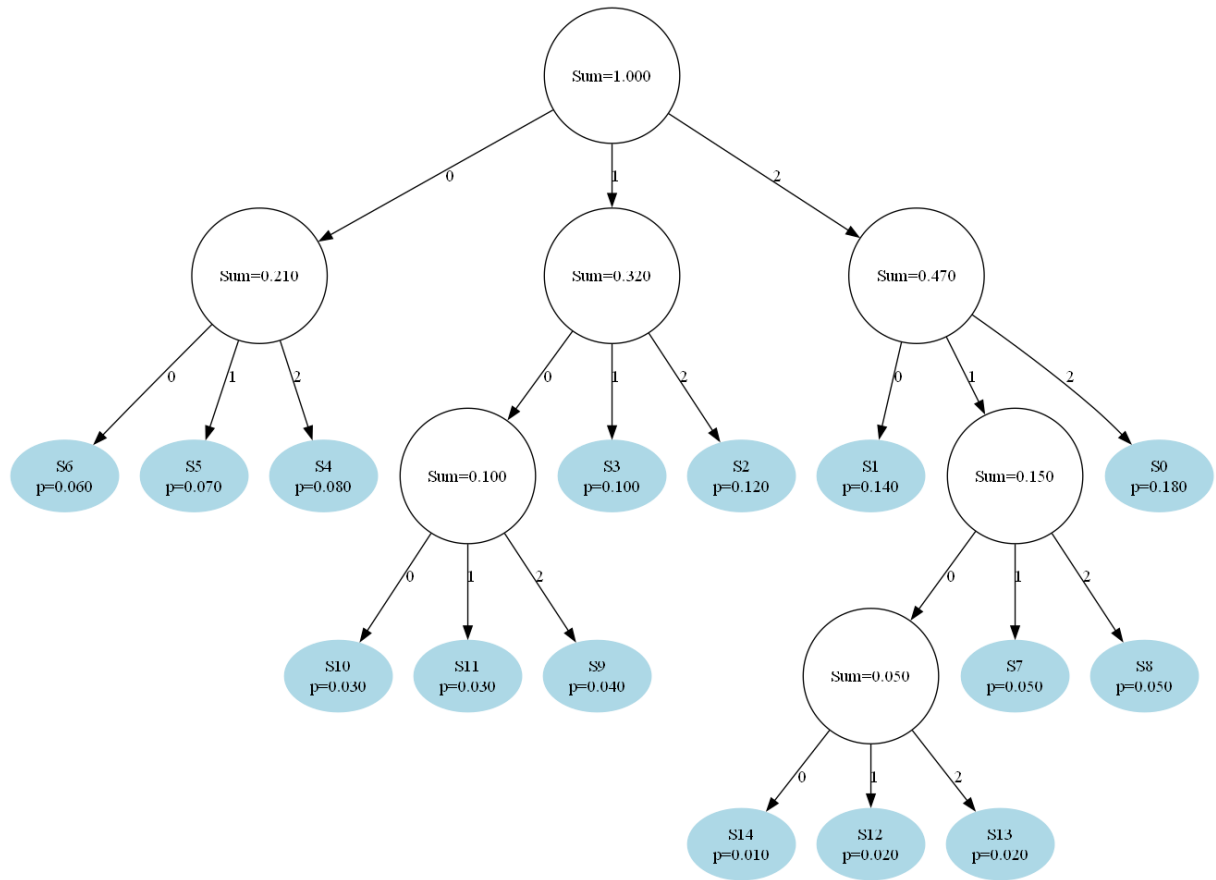
Kraft inequality verified.

<i>Symbol</i>	Huffman	Shannon-Fano	Shannon (Canonical)	Shannon-type
<i>S0</i>	22	00	00	00
<i>S1</i>	20	01	01	01
<i>S2</i>	12	10	02	02
<i>S3</i>	11	11	100	102
<i>S4</i>	02	120	101	112
<i>S5</i>	01	121	102	121
<i>S6</i>	00	200	110	200
<i>S7</i>	211	201	111	202
<i>S8</i>	212	210	112	210
<i>S9</i>	102	211	120	211
<i>S10</i>	100	220	1210	2200
<i>S11</i>	101	221	1211	2202
<i>S12</i>	2101	2220	1212	2211
<i>S13</i>	2102	2221	1220	2220
<i>S14</i>	2100	2222	12210	22220

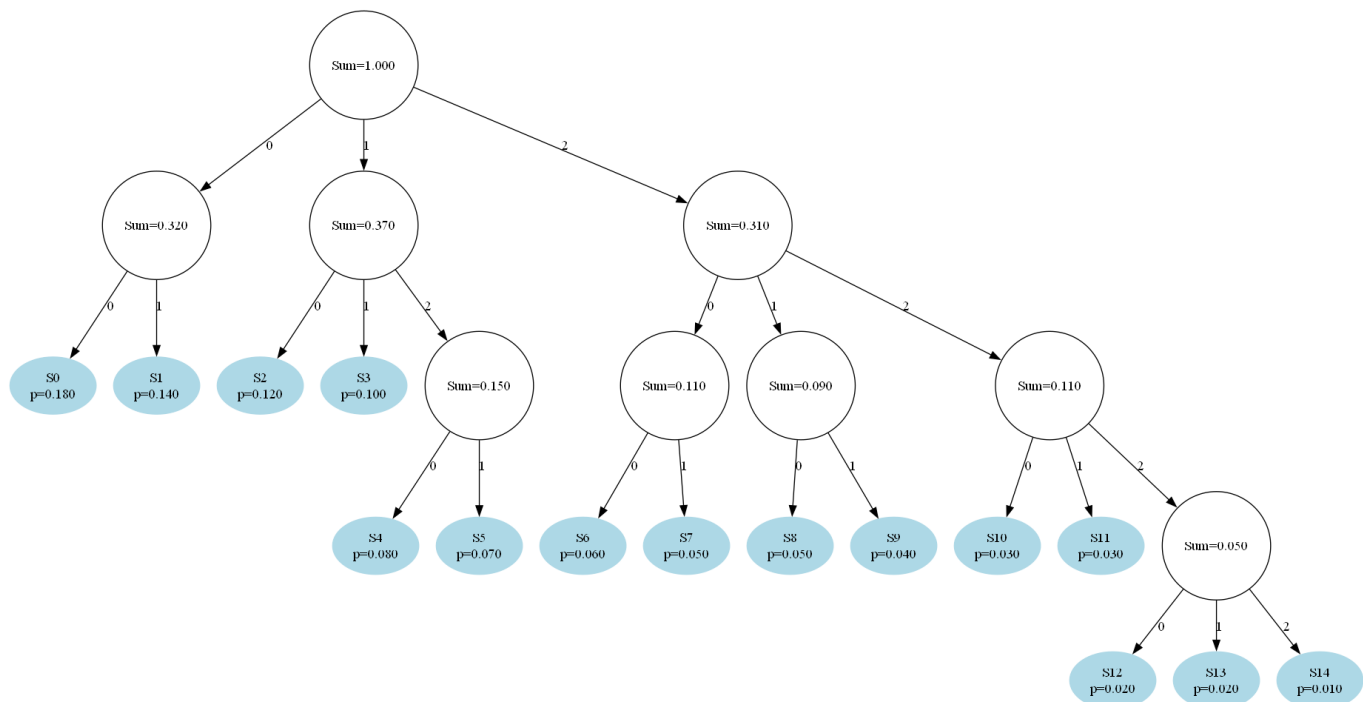
Encoded string (Huffman) to decode: **2220102210011001**

Decoded symbols: ['S0', 'S1', 'S9', 'S14', 'S3', 'S6']

Huffman Tree



Shannon-Fano Tree



4. Conclusion

Huffman coding achieves the minimum expected length and highest efficiency. Shannon-Fano performs moderately well but is not always optimal. Shannon and Shannon-type codes produce identical expected lengths as they use the same length formula. The entropy bound and Kraft inequality are satisfied.

5. Program

```
import math
import heapq
from graphviz import Digraph

# =====
# ----- COMMON UTILITIES -----
# =====

def entropy(symbols, base):
    return -sum(p * math.log(p, base) for p in symbols.values() if p > 0)

def expected_length(symbols, codebook):
    return sum(symbols[s] * len(codebook[s]) for s in symbols)

def kraft_sum(codebook, D):
    return sum(D ** (-len(codebook[s])) for s in codebook)

# =====
# ----- HUFFMAN -----
# =====

class HuffmanNode:
    def __init__(self, value, symbol=None):
        self.value = value
        self.symbol = symbol
        self.children = []

    def __lt__(self, other):
        return self.value < other.value

def huffman(symbols, D):
    heap = []

    for s, p in symbols.items():
        heapq.heappush(heap, HuffmanNode(p, s))

    # Dummy nodes only required for D > 2
    if D > 2:
        n = len(heap)
        remainder = (n - 1) % (D - 1)
        if remainder != 0:
            needed = (D - 1) - remainder
            for i in range(needed):
```

```

        heapq.heappush(heap, HuffmanNode(0, f'dummy{i}'))

while len(heap) > 1:
    smallest = []
    for _ in range(D):
        if heap:
            smallest.append(heapq.heappop(heap))

    parent = HuffmanNode(sum(n.value for n in smallest))
    parent.children = smallest
    heapq.heappush(heap, parent)

return heap[0]

def generate_codes_tree(node, prefix="", codebook=None):
    if codebook is None:
        codebook = {}

    if node.symbol is not None and not node.symbol.startswith("dummy"):
        codebook[node.symbol] = prefix
        return codebook

    for i, child in enumerate(node.children):
        generate_codes_tree(child, prefix + str(i), codebook)

    return codebook

def decode_huffman(encoded, root):
    decoded = []
    current = root

    for digit in encoded:
        current = current.children[int(digit)]
        if current.symbol is not None:
            decoded.append(current.symbol)
            current = root

    return decoded

# =====
# ----- SHANNON-FANO -----
# =====

class FanoNode:
    def __init__(self, value, symbol=None):
        self.value = value
        self.symbol = symbol
        self.children = []

def split_into_d_groups(symbol_list, D):
    groups = []
    remaining = symbol_list.copy()
    remaining_groups = D

    while remaining_groups > 1 and remaining:
        total = sum(p for _, p in remaining)
        target = total / remaining_groups

```

```

current_group = []
current_sum = 0
i = 0

while i < len(remaining):
    symbol, prob = remaining[i]
    if current_sum + prob <= target:
        current_group.append((symbol, prob))
        current_sum += prob
        i += 1
    else:
        if abs((current_sum + prob) - target) < abs(current_sum - target):
            current_group.append((symbol, prob))
            i += 1
        break

if not current_group:
    current_group.append(remaining[0])
    i = 1

groups.append(current_group)
remaining = remaining[i:]
remaining_groups -= 1

if remaining:
    groups.append(remaining)

while len(groups) < D:
    groups.append([])

return groups

def build_fano(symbol_list, D):
    total = sum(p for _, p in symbol_list)
    node = FanoNode(total)

    if len(symbol_list) == 1:
        node.symbol = symbol_list[0][0]
        return node

    groups = split_into_d_groups(symbol_list, D)

    for group in groups:
        if group:
            child = build_fano(group, D)
            node.children.append(child)

    return node

# =====
# ----- SHANNON (CANONICAL) -----
# =====

def shannon(symbols, D):
    lengths = {s: math.ceil(-math.log(p, D) - 1e-12) for s, p in symbols.items()}

    sorted_symbols = sorted(symbols.items(),
                           key=lambda x: (lengths[x[0]], -x[1]))

```

```

codebook = {}
current_code = 0
previous_length = lengths[sorted_symbols[0][0]]

for s, p in sorted_symbols:
    Li = lengths[s]

    if Li > previous_length:
        current_code *= D ** (Li - previous_length)
        previous_length = Li

    code = ""
    temp = current_code
    for _ in range(Li):
        code = str(temp % D) + code
        temp //= D

    codebook[s] = code
    current_code += 1

return codebook

```

```

# =====
# ----- SHANNON-TYPE -----
# =====

```

```

def shannon_type(symbols, D):
    sorted_symbols = sorted(symbols.items(),
                           key=lambda x: x[1], reverse=True)

```

```

codebook = {}
cumulative = 0

```

```

for s, p in sorted_symbols:
    Li = math.ceil(-math.log(p, D) - 1e-12)

```

```

Fi = cumulative
Ci = ""
temp = Fi

```

```

for _ in range(Li):
    temp *= D
    digit = int(temp + 1e-12)
    Ci += str(digit)
    temp -= digit

```

```

codebook[s] = Ci
cumulative += p

```

```

return codebook

```

```

# =====
# ----- GRAPHICAL TREE -----
# =====

```

```

def draw_tree(root, filename="tree", title="Tree"):
    dot = Digraph(comment=title)
    dot.attr(rankdir='TB')

```

```

node_counter = [0]

def add_nodes(node, parent_id=None, edge_label=""):
    node_id = str(node_counter[0])
    node_counter[0] += 1

    if node.symbol is not None:
        label = f"{node.symbol}\np={node.value:.3f}"
        dot.node(node_id, label, shape="ellipse",
                 style="filled", color="lightblue")
    else:
        label = f"Sum={node.value:.3f}"
        dot.node(node_id, label, shape="circle")

    if parent_id is not None:
        dot.edge(parent_id, node_id, label=edge_label)

    for i, child in enumerate(node.children):
        add_nodes(child, node_id, str(i))

add_nodes(root)
dot.render(filename, format="png", cleanup=True)
print(f"{title} saved as {filename}.png")

# =====
# ----- MAIN -----
# =====

if __name__ == "__main__":

    print("\n=== D-ary Source Coding Comparison Program ===")

    D = int(input("Enter D (2=Binary, 3=Ternary, ...): "))
    n = int(input("Enter number of symbols: "))

    symbols = {}
    print("\nEnter probabilities (must sum to 1):")
    for i in range(n):
        p = float(input(f"Probability S{i}: "))
        symbols[f"S{i}"] = p

    H = entropy(symbols, D)

    # Huffman
    h_root = huffman(symbols, D)
    h_codes = generate_codes_tree(h_root)
    L_h = expected_length(symbols, h_codes)

    # Shannon-Fano
    sorted_symbols = sorted(symbols.items(),
                           key=lambda x: x[1], reverse=True)
    f_root = build_fano(sorted_symbols, D)
    f_codes = generate_codes_tree(f_root)
    L_f = expected_length(symbols, f_codes)

    # Shannon
    s_codes = shannon(symbols, D)
    L_s = expected_length(symbols, s_codes)

```



```

# Shannon-type
st_codes = shannon_type(symbols, D)
L_st = expected_length(symbols, st_codes)

# ===== OUTPUT =====

print("\n--- Generated Codes ---")

print("\nHuffman:")
for s in symbols:
    print(f"{s}: {h_codes[s]}")

print("\nShannon-Fano:")
for s in symbols:
    print(f"{s}: {f_codes[s]}")

print("\nShannon (Canonical):")
for s in symbols:
    print(f"{s}: {s_codes[s]}")

print("\nShannon-type:")
for s in symbols:
    print(f"{s}: {st_codes[s]}")

print("\n--- Performance ---")
print(f"Entropy (base {D}) = {H:.4f}\n")

print("Method\tExpected Length\tEfficiency")
print(f"Huffman\t\t{L_h:.4f}\t\t{H/L_h:.4f}")
print(f"Shannon-Fano\t\t{L_f:.4f}\t\t{H/L_f:.4f}")
print(f"Shannon\t\t{L_s:.4f}\t\t{H/L_s:.4f}")
print(f"Shannon-type\t\t{L_st:.4f}\t\t{H/L_st:.4f}")

print("\nEntropy Bound Check ( $H \leq L < H+1$ ):")
print(f"Huffman: {H <= L_h < H+1}")
print(f"Shannon-Fano: {H <= L_f < H+1}")
print(f"Shannon: {H <= L_s < H+1}")
print(f"Shannon-type: {H <= L_st < H+1}")

print("\nKraft Sum Check:")
print(f"Huffman: {kraft_sum(h_codes, D):.4f}")
print(f"Shannon-Fano: {kraft_sum(f_codes, D):.4f}")
print(f"Shannon: {kraft_sum(s_codes, D):.4f}")
print(f"Shannon-type: {kraft_sum(st_codes, D):.4f}")

# Huffman decoding only
encoded = input("\nEnter encoded string (Huffman) to decode: ")
if encoded:
    decoded = decode_huffman(encoded, h_root)
    print("Decoded symbols:", decoded)

# Draw trees
draw_tree(h_root, "huffman_tree", "Huffman Tree")
draw_tree(f_root, "fano_tree", "Shannon-Fano Tree")

```